Failure (prove the suite catches a regression) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `regression_catch_practice.py`: a green test suite only means something if you've actually watched it turn red for a real reason at least once. Deleting one key instruction from a prompt on purpose, then re-running the eval and watching the score genuinely drop, is that proof. **If not:** the Build Task's regression check would be the first time you ever deliberately made a prompt worse — and you'd have no way to tell whether a missing score drop meant "the prompt change was harmless" or "the suite itself is blind."

All versions use this prompt file — create it in `practice/` first:

```
# practice/support_prompt.txt
You are a support assistant for an online shop.
Always mention our 30-day return policy when a customer asks
about returns, refunds or exchanges.
Keep every reply under 60 words.
```

The solution calls the real OpenAI API, and runs from inside `practice/` with `python regression_catch_practice.py`.

Solution

Story: "the score dropped" could just mean one task out of four wobbled randomly, nothing more. Deciding the pass bar — exactly how many passes still counts as OK — *before* the sabotage happens, not after, is what stops you from explaining the result away once you see it. **If not:** any small dip could be read after the fact as "caught it," and a real hole in the suite could just as easily be waved off as noise.

```
# practice/regression_catch_practice.py
from openai import OpenAI

client = OpenAI()

# why: decided BEFORE sabotaging, so the result can't be
# explained away afterwards
```

```
PASS_BAR = 0.75

TASKS = [
    {"id": "return_shoes",
     "question": "Can I return shoes I bought last week?",
     "must_include": "30"},
    {"id": "refund_lamp",
     "question": "How do I get a refund for a broken lamp?",
     "must_include": "30"},
    {"id": "exchange_shirt",
     "question": "Can I exchange a shirt for a bigger size?",
     "must_include": "30"},
    {"id": "send_back_jacket",
     "question": "I want to send back a jacket. What are the rules?",
     "must_include": "30"},
]

def load_prompt() -> str:
    # how: read fresh on every call, so an edit to the file shows up
    # on the very next run without restarting the script
    with open("support_prompt.txt") as f:
        return f.read()

def answer(question: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[
            {"role": "system", "content": load_prompt()},
            {"role": "user", "content": question},
        ],
    )
    return response.choices[0].message.content

def run_eval_suite() -> dict:
    results = {}
    for task in TASKS:
        reply = answer(task["question"])
        results[task["id"]] = task["must_include"] in reply
    return results

def score(results: dict) -> float:
    passed = 0
    for task_id in results:
        if results[task_id]:
            passed = passed + 1
    return passed / len(results)
```

```
# Measure the baseline, sabotage the prompt by hand, measure again, and
# check the suite notices the drop.
baseline = run_eval_suite()
print(f"baseline score: {score(baseline):.2f}")

input("Delete the 30-day line from support_prompt.txt, save, "
      "then press Enter...")

after = run_eval_suite()
print(f"after sabotage score: {score(after):.2f}")

if score(baseline) >= PASS_BAR and score(after) < PASS_BAR:
    print("PASS: the suite caught the regression.")
else:
    print("FAIL: the suite did NOT catch it. Tighten the rules.")
```

Expected output:

```
baseline score: 1.00
Delete the 30-day line from support_prompt.txt, save, then press Enter...
after sabotage score: 0.00
PASS: the suite caught the regression.
```